

RTL Design Sherpa

KESTREL-RV32I Hardware Architecture Specification 0.1

October 7, 2026

Table of Contents

1 Kestrel Has Index....9

2 Document Information....10

2.1 KESTREL-RV32I Hardware Architecture Specification....10

2.2 Document Purpose....11

2.3 References....12

2.4 Terminology....13

2.5 Revision History....14

3 Purpose and Scope....16

3.1 Document Purpose....16

3.2 What kestrel Is....17

3.2.1 Figure 1.1: The falcon ladder, smallest to largest....17

3.3 Scope....18

3.3.1 In Scope....18

3.3.2 Explicit Non-Goals....18

3.4 Intended Audience....20

3.5 Specification Conventions....21

4 System Overview....22

4.1 Top-Level Description....22

4.2 System Context....23

4.2.1 Figure 2.1: kestrel in a minimal system and in a loader-based SoC....23

4.2.2 Minimal Integration (No Loader)....24

4.2.3 Loader Integration (Boards)....24

4.3 Key Features..... 25
5 ISA Scope..... 26
5.1 Instruction-Set Coverage..... 26
5.2 Reserved Encodings and HINTs..... 30
5.3 Misaligned Data Accesses..... 31
6 Programmer-Visible Machine State..... 32
6.1 The State an Integrator Programs Against..... 32
6.2 Consequences for Software..... 33
6.3 The Halt Boundary as State..... 34
7 kestrel_core Port Specification..... 35
7.1 Overview..... 35
7.2 Clock and Reset..... 36
7.3 Instruction Memory (Fetch) Port..... 37
7.4 Data Memory Port..... 38
7.5 Halt Outputs..... 39
7.6 RVFI Retirement Channel..... 40
8 Memory Contract..... 42
8.1 The Contract..... 42
8.2 Timing Rules..... 43
8.3 The Cross-Word Retry..... 44
8.4 What the Contract Deliberately Excludes..... 45
9 Halt and Trap Behavior..... 46
9.1 Halt Versus Trap..... 46
9.2 Halt Causes..... 47

9.3 Timing Contract....48
9.4 Writeback Suppression Detail....49
9.5 ECALL as the Termination Convention....50
10 kestrel_mem_loader and the AXIL Programming Interface....51
10.1 Purpose....51
10.2 Parameters....52
10.3 AXIL Slave Ports....53
10.4 Core-Side Ports and Debug Taps....55
10.5 Address Map....56
10.6 CTRL Register and Run Control....58
10.7 Mutual Exclusion (Load Mode vs Run Mode)....59
10.8 Programming Sequence....60
11 Performance Characteristics....61
11.1 Cycles per Instruction....61
11.2 Critical Path Analysis....62
11.3 Resource Sketch....63
11.4 Throughput Context....64
12 Integration Guide....65
12.1 Embedding Recipe (No Loader)....65
12.2 Embedding Recipe (Loader / Board)....66
12.3 Filelists to Consume....67
12.4 Verification Hooks....68
12.5 Known Limitations....69
13 KESTREL-RV32I Micro-Architecture Specification Index....70

13.1 Document Organization....71
13.1.1 Front Matter....71
13.1.2 Chapter 1: Overview....71
13.1.3 Chapter 2: Functional Blocks....71
13.1.4 Chapter 3: Internal Interfaces....71
13.1.5 Chapter 4: Micro-Architecture Behavior....71
13.1.6 Chapter 5: Verification Hooks....71
13.1.7 Chapter 6: Configuration....71
13.2 Related Documentation....72
14 Simplified RV32I: kestrel....73
14.1 Overview....74
14.2 Document Structure....75
14.2.1 Chapter 1: Introduction....75
14.2.2 Chapter 2: The RV32I ISA Quick Reference....75
14.2.3 Chapter 3: The kestrel Datapath....75
14.2.4 Chapter 4: The Control Truth Table....75
14.2.5 Chapter 5: The Memory Contract....75
14.2.6 Chapter 6: Verification....76
14.2.7 Chapter 7: What kestrel Costs....76
14.2.8 Version History....76
15 RISC-V Falcon Suite....77
15.1 The ladder....78
15.1.1 Moved: hive-serv....79
15.1.2 Tetrad engine: gatling-rv32itetrad....79

15.2 Goals per rung....80
15.3 Packaging (per rung, the MC-line treatment)....81
15.4 References and layout....82
16 kestrel-rv32i Testplans....83
16.1 Inventory....84
16.2 What each plan covers....85
16.3 JUnit naming convention (observed)....86
16.4 Rollup....87
16.5 Running the tests....88
16.6 Functional coverage (CORE-18)....89

List of Figures

Figure 1.1: The falcon ladder, smallest to largest.....17
Figure 2.1: kestrel in a minimal system and in a loader-based SoC.....23

List of Tables

Table 1: Reference Documents....12
Table 2: Terminology and Definitions....13
Table 3: Document Revision History....14
Table 4: Explicit non-goals of kestrel-rv32i....18
Table 5: Intended audience....20
Table 6: kestrel deliverables....22
Table 7: Key features....25
Table 8: ISA support matrix by instruction class....26
Table 9: Programmer-visible machine state....32
Table 10: Halt causes (encodings centralized in rtl/includes/kestrel_pkg.sv)....34
Table 11: Clock and reset....36
Table 12: Instruction memory port....37
Table 13: Data memory port....38
Table 14: Halt outputs....39
Table 15: RVFI retirement channel....40
Table 16: Halt cause encodings (single source of truth: rtl/includes/kestrel_pkg.sv)
....47
Table 17: Loader parameters....52
Table 18: AXIL slave interface....53
Table 19: Core-side and debug ports....55
Table 20: Loader address map (AXIL byte addresses)....56
Table 21: CTRL register (AXIL offset 0x0002_0000)....58
Table 22: CPI by access class....61
Table 23: Critical-path content per instruction class....62
Table 24: Resource sketch (no committed synthesis numbers at v0.1)....63
Table 25: Filelists (KESTREL_ROOT = the kestrel-rv32i directory; REPO_ROOT = the
RTLDesignSherpa checkout root)....67
Table 26: Observability hooks....68
Table 27: Version history....76

1 Kestrel Has Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 KESTREL-RV32I Hardware Architecture Specification

Document Number: KESTREL-HAS-001 **Version:** 0.4 **Status:** Draft **Classification:** Open Source - MIT License

---

2.2 Document Purpose

This Hardware Architecture Specification (HAS) defines the external, system-level architecture of the kestrel-rv32i core: what the core is, what it implements, how it is connected, how it behaves at its boundaries, and how it is integrated into a larger system. It is written for consumers of the core — integrators, firmware engineers, and verification engineers — and is deliberately interface-first: ports, contracts, encodings, and sequences, with rationale where the boundary behavior is a design decision rather than an obvious default.

This document does not describe the internal microarchitecture. Block internals, cycle-level behavior, and implementation rationale live in the companion Micro-Architecture Specification (MAS).

Target Audience: - System architects evaluating kestrel for integration - Hardware engineers embedding the core and its optional board loader - Software engineers writing or porting programs for the core - Verification engineers planning core-level and system-level test coverage

Companion Documents: - KESTREL-RV32I Micro-Architecture Specification (MAS) - Internal block-level implementation - Simplified RV32I: kestrel - Tutorial-study book covering the same ground as pedagogy

---

2.3 References

Reference Documents

ID	Document	Description
[REF-1]	KESTREL MAS v0.1	Micro-Architecture Specification (companion)
[REF-2]	RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (`projects/components/riscv-ip/references/riscv-spec.pdf`)	ISA authority; cited by chapter/section as “unpriv §N”
[REF-3]	riscv-formal `rvfi.md` (`vendor/riscv-formal/docs/rvfi.md`)	RVFI retire-channel field semantics
[REF-4]	KESTREL testplans (`dv/testplans/*.yaml`)	V&V scenarios and coverage cross-reference
[REF-5]	Simplified RV32I: kestrel v0.1 (`docs/simplified_rv32i/`)	Study book; prose source for this specification
[REF-6]	falcon-suite design spec (`docs/superpowers/specs/2026-10-06-riscv-falcon-suite-design.md`)	Suite scope and packaging authority

2.4 Terminology

Terminology and Definitions

Term	Definition
AXIL	AMBA AXI4-Lite - simple single-beat AXI subset used by the board loader
Beat	One memory access cycle; a cross-word access uses two beats
CPI	Cycles per instruction
FUB	Functional Unit Block - leaf RTL module
GPR	General-purpose register (x0-x31; x0 is hard-wired zero)
HALT_*	kestrel halt-cause encodings defined in `kestrel_pkg` (see Table 4.3)
Harvard	Separate instruction and data memory ports
IALIGN	Instruction alignment: 32 bits for RV32I without the C extension
Loader	`kestrel_mem_loader` - optional AXIL-slave board glue with on-chip memories
RVFI	RISC-V Formal Interface - one retire record per retired instruction
Trap beat	The single RVFI beat of a halting instruction (`rvfi_trap=1`)
Unified map	Loader address map in which one address bit selects the memory array on every port

2.5 Revision History

Document Revision History

Version	Date	Author	Description
0.1	2026-10-07	seang	Initial HAS release; documents the as-built RTL (Tasks 1-11) including the Task-9 IALIGN halt and the Task-11 AXIL board loader
0.2	2026-10-20	seang	Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.
0.3	2026-10-20	seang	Section (h2) headings now open a new page, the pipeline default. No technical content changed.
0.4	2026-10-20	seang	Memory contract: registered-read memories may hold the core with the new `mem_wait` input (owner, 2026-10-20); port table and timing rules updated. Combinational memories tie it low and see the unchanged single-cycle core.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 Purpose and Scope

3.1 Document Purpose

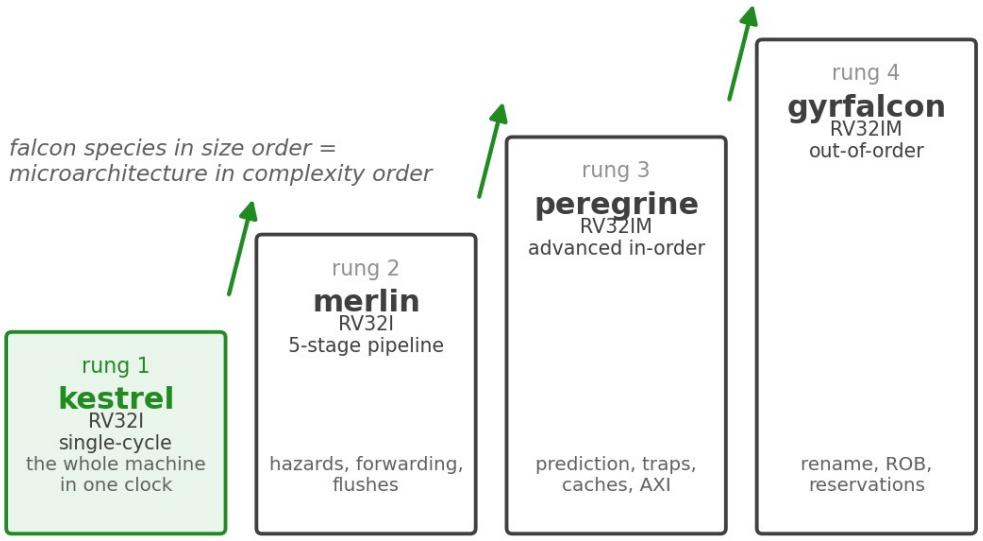
This Hardware Architecture Specification is the integration contract for the kestrel-rv32i core. It specifies:

1. **System architecture** - what the core is, what surrounds it, and how the pieces connect
2. **ISA scope** - exactly which instructions and instruction classes the core implements, and how it treats the classes it does not
3. **External interfaces** - complete port specifications for `kestrel_core` and the optional `kestrel_mem_loader`, including the AXIL programming interface
4. **Boundary behavior** - the memory contract, the halt contract, and the retirement-report (RVFI) contract
5. **Performance characteristics** - cycles per instruction and what limits the clock
6. **Integration requirements** - embedding recipes, filelists, verification hooks, and known limitations

3.2 What kestrel Is

kestrel-rv32i (“kestrel”) is a single-cycle RV32I processor core: one instruction retires per clock, and the entire machine — fetch, decode, register read, execute, memory access, writeback — completes inside that clock. It is rung 1 of the RISC-V falcon suite, the suite’s deliberately simplest machine, and its role is to exhibit everything the ISA promises and nothing the microarchitecture adds.

3.2.1 Figure 1.1: The falcon ladder, smallest to largest



The falcon ladder: four rungs from single-cycle to out-of-order

Each falcon-suite rung teaches exactly one layer of computer-architecture machinery, and each rung reuses the rung below it. kestrel’s decode truth table becomes rung 2’s per-stage control; rung 3 replaces kestrel’s halt-with-cause stub with real trap delivery; rung 4 adds out-of-order execution. kestrel documents its simplifications honestly because the later rungs are where those simplifications are replaced.

3.3 Scope

3.3.1 In Scope

- RV32I base integer instruction set: all 37 instructions (unpriv §2.1)
- Documented, bounded behavior for the FENCE/FENCE.I and SYSTEM opcode classes
- Hardware misaligned load/store handling, including the two-cycle cross-word retry
- The halt contract: four halt causes, trap-beat retirement, post-halt hold
- The RVFI retirement-report channel
- The optional `kestrel_mem_loader` board glue: AXIL slave, on-chip memories, load-then-run control

3.3.2 Explicit Non-Goals

The following are **not** provided, and integrators must not rely on them:

Explicit non-goals of kestrel-rv32i

Non-goal	What kestrel does instead
M extension (multiply/divide)	Illegal instruction (halt cause `0xF`)
C extension (compressed instructions)	IALIGN=32 only; C encodings are illegal instructions
Real CSR support	Bounded CSR stub: reads return zero, writes drop, retired as a documented stub (not CSR support)
Real trap/exception handling	Halt: the core stops with a cause code; no trap vector, no `mepc`/`mtvec`, no privilege transfer
Interrupts (M/S/U)	Nothing; no interrupt inputs exist
MMU / virtual memory / paging	None; all addresses are physical, memory protection does not exist
PMP, counters, debug mode	None
Misaligned instruction fetch	Never executed; a taken control transfer to a non-4-aligned target halts (cause `0x3`)

The suite’s rung 3 (peregrine) is where real trap handling, a machine-mode CSR file, and caches enter. kestrel keeps the seam visible on purpose.

3.4 Intended Audience

Intended audience

Audience	Primary use of this document
System architects	Decide whether kestrel fits the system; understand the contracts
Hardware engineers	Embed the core; wire the loader; close timing against the memory contract
Software/ firmware engineers	Link images for the core; understand halt semantics and the CSR stub boundary
Verification engineers	Plan core-level and system-level coverage against the testplans

3.5 Specification Conventions

- Signal and parameter names appear in code font exactly as in the RTL (`dmem_wstrb`, `RESET_ADDR`) so every claim can be grepped.
- Hexadecimal values use `0x` or SystemVerilog sizing (`32'hFFFF_FFFE`); encodings use sized binary (`7'b0110011`).
- Instruction mnemonics are uppercase (ADD, JALR); assembly in prose is lowercase.
- The RISC-V specification is cited by chapter and section (`unpriv §2.1.6`), never by page; the pinned PDF at `projects/components/riscv-ip/references/riscv-spec.pdf` is the authority.
- **Halt versus trap** is used the way the core uses its ports: a *trap* is a control transfer to a handler (kestrel never traps); a *halt* is kestrel's stop condition — PC frozen, writeback suppressed, cause code on `halt_cause`.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 System Overview

4.1 Top-Level Description

kestrel is delivered as two synthesizable modules and a shared package:

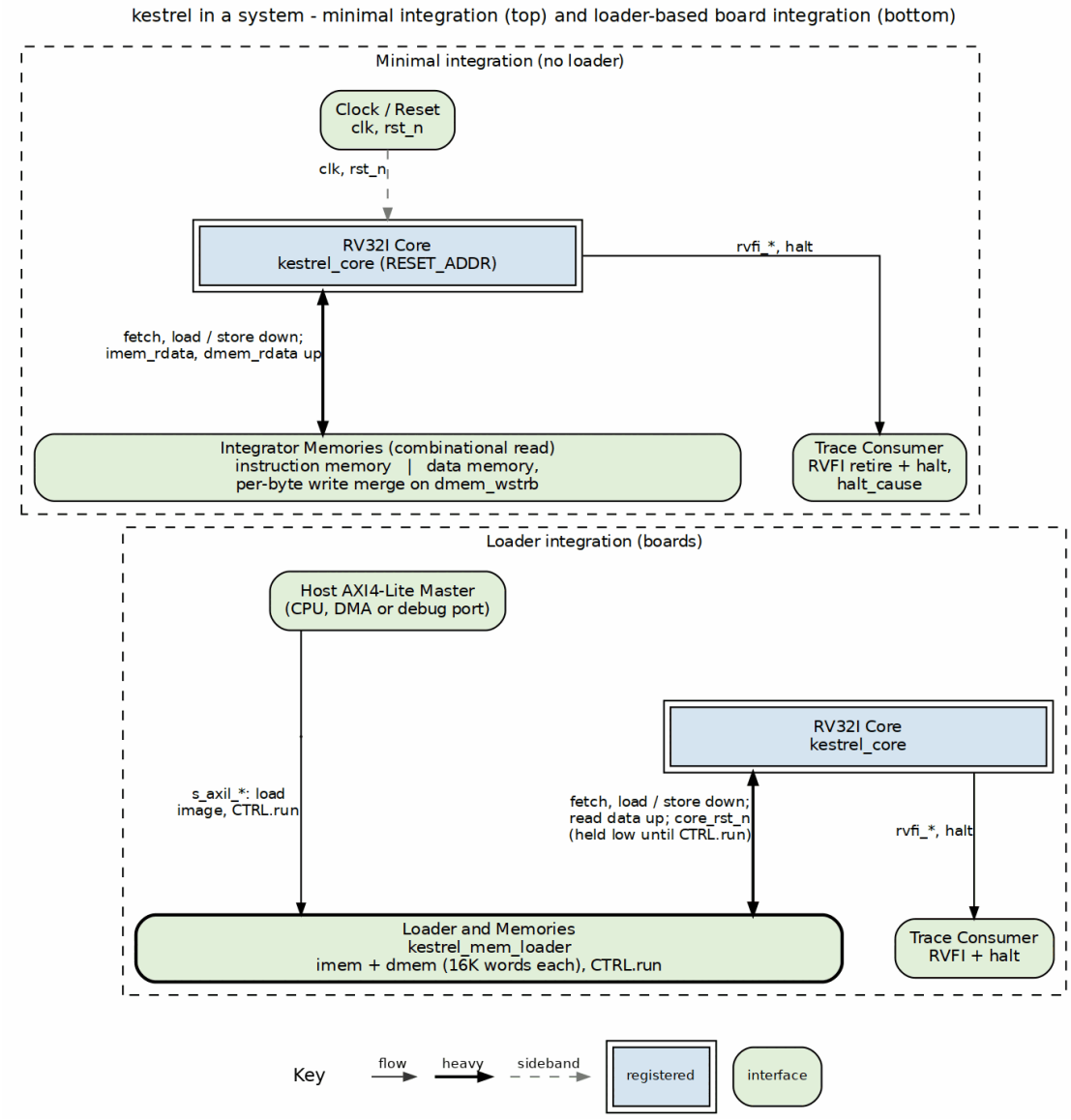
kestrel deliverables

Deliverable	File	Role
`kestrel_pkg`	`rtl/includes/kestrel_pkg.sv`	Shared enums and the halt-cause encodings (single source of truth)
`kestrel_core`	`rtl/top/kestrel_core.sv`	The RV32I core: fetch port, data port, halt outputs, RVFI retire port
`kestrel_mem_loader`	`rtl/fub/kestrel_mem_loader.sv`	Optional board glue: two 64 KB on-chip memories behind an AXIL slave, with load-then-run control of the core

The core has no memory inside it. Integration therefore always means: core + instruction memory + data memory, where both memories must honor the combinational-read contract of Chapter 4. The loader exists for boards: it implements both memories as distributed-RAM arrays and lets a host load a program over AXIL before releasing the core from reset.

4.2 System Context

4.2.1 Figure 2.1: kestrel in a minimal system and in a loader-based SoC



kestrel system context: minimal integration and loader-based integration

The top region shows the minimal integration: `kestrel_core` with two independently provided memories. The bottom region shows the loader integration: `kestrel_mem_loader` supplies both memories and a run-control register; a host agent on the SoC interconnect (a CPU, a DMA engine, or a debug port) drives the loader's AXIL slave to load the image and start the core.

4.2.2 Minimal Integration (No Loader)

Provide: - A 32-bit-wide instruction memory with combinational read (`imem_addr` in, `imem_rdata` out, same cycle) - A 32-bit-wide data memory with combinational read and a per-byte write merge honoring `dmem_wstrb` (word write `dmem_wdata`)

This is the integration used by the simulation testbench `dv/tb/kestrel_tb_top.sv` (with a unified 64 Ki-word array serving both ports so self-modifying code works with no coherence path — a testbench choice, not a core feature).

4.2.3 Loader Integration (Boards)

Provide: - One clock (`aclk`) and one active-low reset (`aresetn`) - An AXIL master able to write the loader's address map (AW/W/B and AR/R channels) - Optionally, observation of `core_rst_n` and the two `o_dbg_busy_*` taps

The loader holds the core in reset while `CTRL.run == 0`, takes the program image over AXIL, and releases the core when software writes `CTRL.run = 1`. See Chapter 4 for the address map and programming sequence.

4.3 Key Features

Key features

Feature	Value
ISA	RV32I base integer, all 37 instructions (unpriv §2.1)
Microarchitecture	Single-cycle; no pipeline, no FSM in the datapath
Memory interface	Harvard: separate 32-bit fetch and data ports, combinational reads
Misaligned data	Handled in hardware; cross-word accesses take a 2-cycle retry
Exceptions	None delivered; four halt causes on `halt`/`halt_cause` ports
Observability	First-class RVFI retire channel (17 signals) plus halt ports
Board support	Optional AXIL loader with 64 KB imem + 64 KB dmem and run control
Configuration	One core parameter (`RESET_ADDR`); eight loader parameters

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

5 ISA Scope

5.1 Instruction-Set Coverage

kestrel implements the complete RV32I base integer instruction set — all 37 instructions — plus documented, bounded behavior for the FENCE/FENCE.I and SYSTEM opcode classes that surround the base set. The table below is the support matrix an integrator codes against; per-instruction encodings are the authority of the ISA manual (unpriv §2.1) and the decode truth table in the MAS.

ISA support matrix by instruction class

Instruction class	Instructions	Support	Notes
OP (R-type ALU)	ADD SUB SLL SLT SLTU XOR SRL SRA OR AND	Full	funct7 must match; shift funct7 mismatches are illegal
OP-IMM (I-type ALU)	ADDI SLTI SLTIU XORI ORI ANDI SLLI SRLI SRAI	Full	SLTIU compares against the sign-extended immediate, unsigned (unpriv §2.1.4.1)
Loads	LB LH LW LBU LHU	Full	Effective address `rs1 + sext(imm)`; sign/zero extension per size

Instruction class	Instructions	Support	Notes
Stores	SB SH SW	Full	Byte strobes rotated into position; see memory contract
Branches	BEQ BNE BLT BGE BLTU BGEU	Full	Taken and not-taken both cost one cycle
Jumps	JAL JALR	Full	JALR masks target bit 0 (`unpriv §2.1.5.1`)
Upper immediates	LUI AUIPC	Full	LUI writes the immediate directly (never through the ALU)
FENCE / FENCE.I	FENCE FENCE.I	Retire as NOPs	No ordering hardware exists; MISC-MEM funct3 2-7 are illegal

Instruction class	Instructions	Support	Notes
ECALL / EBREAK	ECALL EBREAK	Halt	Causes `0x1` / `0x2`; the halting instruction retires as an RVFI trap beat
MRET	MRET	Retire as NOP (pc+4 fall-through)	The riscv-tests p-environment points `mepc` at the next instruction; pc+4 is what the test expects. This is not trap support
CSR class (Zicsr forms)	CSRRW CSRRS CSRRC CSRRWI CSRRSI CSRRCI	Bounded stub	`csr_stub`: reads return zero, writes drop, rd writes 0. **This is not CSR support** — there is no CSR

Instruction class	Instructions	Support	Notes
			file, no `mstatus`, no `mtvec`
SYSTEM reserved	SYSTEM funct3 100; SYSTEM imm12 other than ECALL/EBREAK/MRET	Illegal	Halt cause `0xF`
M/A/C extensions	—	Not present	These encodings fall to the illegal-instruction default

5.2 Reserved Encodings and HINTs

Every encoding not claimed by a decode row halts as an illegal instruction (cause `0xF`): branch funct3 2-3, load/store funct3 3 and 6-7, MISC-MEM funct3 2-7, SYSTEM funct3 100, SYSTEM imm12 outside ECALL/EBREAK/MRET, and shift funct7 mismatches (unpriv §§2.1.4, 2.1.6 reserve these). HINT encodings — base instructions with rd = x0 (unpriv §2.1.9) — execute as their base instruction; the x0 write is discarded, so the architectural effect is the NOP the spec recommends.

5.3 Misaligned Data Accesses

RV32I permits, but does not require, hardware handling of misaligned loads and stores (unpriv §2.1.6). kestrel handles them in hardware: byte-lane rotation absorbs any in-word misalignment in the single cycle, and an access that crosses a word boundary (`addr[1:0] + size > 4`) takes a documented two-cycle retry — the PC holds for one cycle, a second beat issues at the next word, and the instruction retires once. Fixed cost: one cycle in-word, two crossing. A cross-word access is two word reads/writes and is not atomic, which the spec never promised.

Instruction-side misalignment is not permitted this latitude: with IALIGN=32, a taken branch, JAL, or JALR whose resolved target is not 4-byte aligned halts the core (cause `0x3`).

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 Programmer-Visible Machine State

6.1 The State an Integrator Programs Against

The architectural state visible to software is exactly:

Programmer-visible machine state

State element	Width	Reset value	Description
PC	32	`RESET_ADDR` (parameter, default `32'h0000_0000`)	Program counter; fetch address of the currently retiring instruction
x0	32	0	Hard-wired zero; reads return 0, writes are discarded in the register file
x1-x31	32 each	0	General-purpose registers; the file resets to all zeros
Memory	integrator-defined	integrator-defined	Flat byte-addressed space via the data port; the core provides no protection, mapping, or ordering semantics
Halt state	1 + 4	`halt=0`, cause `0x0`	Stop flag and cause code on the boundary (below)

There are no CSRs, no counters, no privilege state, and no trap state. `mepc`, `mtvec`, `mstatus` and friends do not exist; the CSR instruction class retires through a stub (Chapter 3) whose only architectural effect is writing zero to rd.

6.2 Consequences for Software

- **Program termination is a halt.** Programs end with ECALL (cause `0x1`); the harness reads the verdict from x3 (`gp`, the register the riscv-tests p-environment uses) at the final writeback before the halt.
- **The first fetch is `RESET_ADDR`.** Images must be linked so execution begins there; the loader integration must set the core's `RESET_ADDR` parameter to the image link base.
- **Registers start at zero.** No startup register has architectural garbage, and the register file reset makes pre-load observability deterministic.
- **Memory needs no setup.** There is no TLB, no cache to enable, no memory attributes: an address is data memory, full stop.

6.3 The Halt Boundary as State

`halt` and `halt_cause[3:0]` are outputs, but software-visible behavior makes them part of the machine state: when any halt condition occurs, the PC freezes, writeback is suppressed, and retirement stops after one final trap beat on RVFI. The four causes:

Halt causes (encodings centralized in `rtl/includes/kestrel_pkg.sv`)

Cause	Name	Raised by	Spec counterpart (unpriv)
`4'h1`	ECALL	decode, ECALL	environment-call exception
`4'h2`	EBREAK	decode, EBREAK	breakpoint exception
`4'h3`	IALIGN	core, taken branch/JAL/JALR to a non-4-aligned target	instruction-address-misaligned exception
`4'hF`	ILL	decode, illegal encoding	illegal-instruction exception

The full halt timing contract — trap beat, post-hold sampling, and the absence of trap recovery — is specified in Chapter 4.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 kestrel_core Port Specification

7.1 Overview

`kestrel_core` (`rtl/top/kestrel_core.sv`) is the deliverable top. All ports are single-clock (`clk`, active-low `rst_n`); there are no clock enables and no bus handshakes. One input, `mem_wait`, lets a memory system with registered reads hold the core (2026-10-20, next section); tied low, the core is the single-cycle machine this chapter describes. One parameter, `RESET_ADDR`, selects the fetch address out of reset.

7.2 Clock and Reset

Clock and reset

Port	Direction	Width	Description
`clk`	Input	1	System clock; the whole machine retires in one `clk` period
`rst_n`	Input	1	Active-low reset (asynchronous assert, synchronous deassert via `ALWAYS_FF_RST`); PC → `RESET_ADDR`, halt/run state cleared, register file cleared
`mem_wait`	Input	1	Memory not ready this cycle: every piece of architectural state holds (PC, register write, halt latch, retry bits, retire counter), `rvfi_valid` is low, and the combinational halt is masked. Tie low for combinational memories; `kestrel_mem_bram` drives it

7.3 Instruction Memory (Fetch) Port

Instruction memory port

Port	Direction	Width	Description
`imem_addr`	Output	32	Fetch address, driven directly by the PC; always 4-byte aligned in operation
`imem_rdata`	Input	32	Instruction word at `imem_addr`, valid in the same cycle (or in the first cycle `mem_wait` is low)

7.4 Data Memory Port

Data memory port

Port	Direction	Width	Description
`dmem_req`	Output	1	A load or store is accessing data memory this cycle
`dmem_we`	Output	1	The access is a store (high) or a load (low)
`dmem_addr`	Output	32	Word-aligned byte address: `{alu_y[31:2], 2'b00}`; on the retry cycle the next word
`dmem_wstrb`	Output	4	Per-byte write enables, rotated into position for stores; all-zero for loads
`dmem_wdata`	Output	32	Store data rotated into the addressed byte lanes
`dmem_rdata`	Input	32	Combinational read data for loads, valid in the same cycle

7.5 Halt Outputs

Halt outputs

Port	Direction	Width	Description
`halt`	Output	1	High on the halting cycle and held high forever after (latched); freezes the PC and stops retirement
`halt_cause`	Output	4	Cause encoding for the halt in progress (`4'h1`, `4'h2`, `4'h3`, `4'hF`); see the halt table in the machine-state chapter

7.6 RVFI Retirement Channel

The RVFI port implements the riscv-formal retire convention (`vendor/riscv-formal/docs/rvfi.md`) with `NRET=1`. Exactly one beat retires per instruction; the halting instruction produces one final beat with `rvfi_trap=1`; the first cycle of a cross-word access produces no beat.

RVFI retirement channel

Port	Direction	Width	Aggregation rule
`rvfi_valid`	Output	1	`rst_n & ~halt_q & ~ls_first & ~mem_wait` — high exactly on retiring cycles
`rvfi_order`	Output	64	Retirement index: 0, 1, 2, … in program order
`rvfi_pc_rdata`	Output	32	PC of the retiring instruction
`rvfi_pc_wdata`	Output	32	Architecturally next PC (`next_pc`); on a cause-`0x3` halt this is the misaligned target
`rvfi_insn`	Output	32	The instruction word
`rvfi_trap`	Output	1	`halt_now`: high on the single halting trap beat
`rvfi_rs1_addr` / `rvfi_rs2_addr`	Output	5 + 5	Register-file read addresses (`insn[19:15]`, `insn[24:20]`)
`rvfi_rs1_rdata` / `rvfi_rs2_rdata`	Output	32 + 32	Register-file read data; x0 reads zero naturally
`rvfi_rd_addr` / `rvfi_rd_wdata`	Output	5 + 32	Writeback destination and data; both zero when no register write retires (including all writes to x0 and all halted cycles)
`rvfi_mem_addr`	Output	32	Unaligned byte address of a load/store (`alu_y`); zero otherwise

Port	Direction	Width	Aggregation rule
`rvfi_mem_rmask` / `rvfi_mem_wmask`	Output	4 + 4	Byte lanes touched, packed from bit 0 relative to `rvfi_mem_addr` (a word access anywhere reports `4'b1111`)
`rvfi_mem_rdata` / `rvfi_mem_wdata`	Output	32 + 32	Assembled read data (loads) / size-truncated store data (`rs2_data & size_mask`); zero otherwise

The invariants verification checks on every run: `rvfi_order` strictly counts 0,1,2,…; first `rvfi_pc_rdata` after reset equals `RESET_ADDR`; `rvfi_rd_wdata == 0` whenever `rvfi_rd_addr == 0`; memory fields are non-zero only on the final cycle of an access.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 Memory Contract

8.1 The Contract

kestrel talks to memory through two ports that are deliberately minimal. There is no bus fabric, no wait states, no byte enables on the instruction side, and no burst machinery. At this rung the memory system is expected to keep up with the core, and the contract says so out loud. An integrator replacing the memories must preserve all three properties below, or use the one sanctioned exception: holding the core with `mem_wait` (property 2).

1. **Harvard at the core.** Fetch and data access are separate ports backed by whatever the integrator provides. The core never issues a data access to the instruction memory or vice versa, and it never sees coherence between them.
2. **Combinational reads, or an explicit wait.** Both memories answer in the same cycle the address is presented. This is the load-bearing simplification: an instruction's full path — fetch, decode, register read, ALU, data read, writeback — must fit in one clock. `kestrel_mem_loader` honors it with `(* ram_style = "distributed" *)` arrays. A memory with registered reads (block RAM) may instead raise `mem_wait` until its data is valid (owner, 2026-10-20): the core holds every piece of state while `mem_wait` is high, so the ISA is unchanged and only the cycle count grows. `kestrel_mem_bram` is that memory system: CPI 2, or 3 for a load. riscv-formal proves the core under arbitrary wait patterns (formal/kestrel/riscv-formal drives `mem_wait` freely, at most two waits in a row).
3. **Word-wide buses with byte strobes.** Data memory is 32 bits wide. Sub-word stores rotate data and strobe into the addressed byte lanes; sub-word loads rotate read data back before size/sign selection. The memory must merge `dmem_wstrb` per byte (write only the strobed bytes of `dmem_wdata`).

8.2 Timing Rules

- An instruction that touches memory completes in exactly one clock when its access lies within one word; a cross-word access takes two clocks via the documented retry (below). With `mem_wait` low these are the only exceptions; each `mem_wait` cycle adds one clock and changes nothing else.
- The PC holds only for the retry's first beat, for halts, and while `mem_wait` is high.
- A store is visible to the very next cycle's load (a cross-word store immediately followed by the load of the same address returns the full assembled value — pinned by testplan scenario CORE-11).
- `dmem_req` qualifies the access; `dmem_we` distinguishes store from load. Loads drive `dmem_addr` and sample `dmem_rdata` in the same cycle; stores drive `dmem_addr`, `dmem_wstrb`, `dmem_wdata` and commit on the clock edge.

8.3 The Cross-Word Retry

When `addr[1:0] + access_size > 4`, the access straddles two words and one 32-bit beat cannot carry it. The core splits it:

1. **Beat 1 (`ls_first`).** Address is the aligned word containing the access start. Stores write the tail bytes (`4 - addr[1:0]` of them) with the left-rotated strobe; loads capture the shifted first word. The PC holds; no retirement (`rvfi_valid` low).
2. **Beat 2 (`ls_second`).** Address is the next word (`word + 1`). Stores write the remaining head bytes with a head-only strobe and right-rotated data; loads merge the captured tail with the second word's head into the assembled value. The instruction retires now, with exactly one RVFI beat.

During both beats `dmem_req` (and `dmem_we` for stores) stays high; `dmem_addr` steps by one word. Everything else in the core is unaware the retry happened.

8.4 What the Contract Deliberately Excludes

- **No instruction-side data access.** Self-modifying code is invisible to the core unless the memory system unifies the ports. The simulation testbench unifies them (one array serving both fetch and load/store) so `rv32ui-p-fence_i` runs without a coherence path — a testbench choice, not a core feature. The loader provides the same unification by address (Chapter 6).
- **No misaligned instruction fetch.** IALIGN=32 keeps fetch 4-byte aligned; the low PC bits are guaranteed by the next-PC logic plus the cause-`0x3` halt.
- **No memory protection, ordering, or atomicity.** RVWMO (unpriv ch. 3) is satisfied vacuously by a core that issues at most one access at a time and never reorders. Cross-word accesses are not atomic.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 Halt and Trap Behavior

9.1 Halt Versus Trap

Every event RV32I routes to a trap — environment call, breakpoint, illegal instruction, instruction-address-misaligned — stops kestrel instead: the core has no trap machinery, no trap vector, no `mepc`/`mtvec`, and no privilege transfer. The suite reserves real trap delivery for rung 3 (peregrine). What kestrel guarantees instead is a precise, observable stop, suitable as a program-termination and error-detection contract.

9.2 Halt Causes

Halt cause encodings (single source of truth: `rtl/includes/kestrel_pkg.sv`)

Cause	Name	Raised in	Trigger	Spec counterpart (unpriv)
`4'h0`	`HALT_NONE`	—	Running (no halt)	—
`4'h1`	`HALT_ECALL`	`kestrel_decode`	ECALL (SYSTEM funct3 000, imm12 000)	environment-call exception (§2.1.8)
`4'h2`	`HALT_EBREAK`	`kestrel_decode`	EBREAK (SYSTEM funct3 000, imm12 001)	breakpoint exception (§2.1.8)
`4'h3`	`HALT_IALIGN`	`kestrel_core`	Taken branch/JAL/JALR resolving to `next_pc[1:0] != 2'b00`	instruction-address-misaligned exception (§2.1.5)
`4'hF`	`HALT_ILL`	`kestrel_decode`	Any encoding no decode row claims (default row)	illegal-instruction exception

Cause ownership is deliberate: decode raises causes 1, 2, and F because it can see the encoding; only the core can raise cause 3 because the condition needs the resolved next PC and the branch decision, which decode never evaluates.

9.3 Timing Contract

1. **Detection cycle.** `halt_now` is the combinational OR of the decode cause and the core's `misalign_target`. The outputs `halt` and `halt_cause` reflect it immediately (`halt = halt_q | halt_now`), so the first halting cycle is visible without waiting for a latch.
2. **Trap beat.** The halting instruction retires on the detection cycle as the final RVFI beat: `rvfi_valid=1`, `rvfi_trap=1`, no rd writeback, no memory fields. On a cause-3 halt, `rvfi_pc_wdata` carries the misaligned target — exactly what a real trap would record as the faulting address.
3. **Hold.** `halt_q` latches on the detection cycle. Afterwards, forever: the PC is frozen, writeback is suppressed, `rvfi_valid` is low, and `halt` stays raised with the cause held on `halt_cause`. Verification samples at least four post-halt cycles to pin the hold (testplan scenario CORE-12).
4. **No recovery.** The only exit from halt is reset. There is no resume address, no handler, no continue.

9.4 Writeback Suppression Detail

JAL/JALR encodings set decode's `rd_wen` (they are link instructions), and a cause-3 halt retires a JAL/JALR encoding — so the writeback gate is `halt` itself, not decode's `rd_wen`: `rd_wen_eff = rd_wen & ~ls_first & ~halt`. The same `halt` gate drives the RVFI rd fields to zero on the trap beat. A taken, aligned JAL/JALR writes `pc+4` normally.

9.5 ECALL as the Termination Convention

In practice ECALL is the program-termination signal: every riscv-tests program ends in the p-environment's `RVTEST_PASS`/`RVTEST_FAIL` sequence whose final ECALL halts the core. Because the halt precedes any trap-vector `sw gp, tohost`, kestrel never writes the tohost mailbox itself; the harness watches the store port for it anyway and reads the verdict from x3's last writeback (1 = pass). Integrators building loaders should treat ECALL-halt with `gp == 1` as the success contract.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 kestrel_mem_loader and the AXIL Programming Interface

10.1 Purpose

`kestrel_mem_loader` (`rtl/fub/kestrel_mem_loader.sv`) is the board glue: it wraps the core with two 64 KB on-chip memories (imem, dmem), an AXIL4 slave for image loading, and a run-control register that sequences load-then-run. It composes the repo's skid-buffered AXIL leaf slaves (`axil4_slave_wr`, `axil4_slave_rd`) exactly as `rtl/amba/shared/sdpram_slave_axil_axil.sv` does. Mutual exclusion between the loader and the core is by protocol — there is no contention arbiter and no FSM in the datapath.

10.2 Parameters

Loader parameters

Parameter	Type	Default	Description
`AXIL_ADDR_WIDTH`	int	18	AXIL address width; bits [17:0] of the map below
`AXIL_DATA_WIDTH`	int	32	AXIL data width (one 32-bit word per beat)
`MEM_WORDS`	int	16384	Words per region: 64 KB imem + 64 KB dmem
`SKID_DEPTH_AW` / `SKID_DEPTH_W` / `SKID_DEPTH_B`	int	2 / 2 / 2	Write-channel skid depths in the `axil4_slave_wr` leaf
`SKID_DEPTH_AR` / `SKID_DEPTH_R`	int	2 / 4	Read-channel skid depths in the `axil4_slave_rd` leaf

10.3 AXIL Slave Ports

AXIL slave interface

Port	Direction	Width	Description
`aclk`	Input	1	Clock; one clock domain for the whole loader + core
`aresetn`	Input	1	Active-low reset (load mode re-enters on reset)
`s_axil_awaddr`	Input	18	Write address (see map)
`s_axil_awprot`	Input	3	Write protection (ignored)
`s_axil_awvalid` / `s_axil_awready`	Input / Output	1 / 1	Write address handshake
`s_axil_wdata`	Input	32	Write data
`s_axil_wstrb`	Input	4	Byte strobes — honored per byte on loader writes
`s_axil_wvalid` / `s_axil_wready`	Input / Output	1 / 1	Write data handshake
`s_axil_bresp`	Output	2	Write response — always `OKAY` (2'b00)
`s_axil_bvalid` / `s_axil_bready`	Output / Input	1 / 1	Write response handshake
`s_axil_araddr`	Input	18	Read address (see map)
`s_axil_arprot`	Input	3	Read protection (ignored)
`s_axil_arvalid` / `s_axil_arready`	Input / Output	1 / 1	Read address handshake

Port	Direction	Width	Description
s_axil_rdata	Output	32	Read data — memory word, or CTRL (run_q) readback
s_axil_rresp	Output	2	Read response — always OKAY (2'b00)
s_axil_rvalid / s_axil_rready	Output / Input	1 / 1	Read data handshake

10.4 Core-Side Ports and Debug Taps

Core-side and debug ports

Port	Direction	Width	Description
`imem_addr` / `imem_rdata`	Input / Output	32 / 32	Core fetch port into the loader's unified map
`dmem_req` / `dmem_addr` / `dmem_rdata` / `dmem_wstrb` / `dmem_wdata`	Input / Input / Output / Input / Input	1 / 32 / 32 / 4 / 32	Core data port into the loader's unified map; stores merge per byte like the TB store port
`core_rst_n`	Output	1	Core reset: low in load mode (core held), high once `CTRL.run` is written
`o_dbg_busy_wr` / `o_dbg_busy_rd`	Output	1 / 1	Leaf-slave busy taps (debug observability; used by the backpressure scenario ML-08)

10.5 Address Map

Byte address bits [17:0]. The array word index is the byte address's low 14 bits, `addr[13:0]`, on every port (AXIL and core) — one convention, so an AXIL write and a core access at the same byte address always touch the same word.

Loader address map (AXIL byte addresses)

Address bits	Meaning
`addr[17] = 1`	CTRL register; a write with `wstrb[0] & wdata[0]` sets `run` (sticky until reset). Read returns `{31'b0, run}`. CTRL occupies `0x0002_0000`
`addr[17] = 0`, `addr[16] = 0`	imem array selected; `addr[13:0]` is the word index
`addr[17] = 0`, `addr[16] = 1`	dmem array selected; `addr[13:0]` is the word index

Address-map properties an integrator must know:

- **Unified array select.** Every port — AXIL and both core ports — selects imem vs dmem by address bit 16, so the core's fetch port can read the dmem array and its data port can reach either array. A flat image (code, `.tohost`, and `.data` interleaved by address, as the riscv-tests p-environment links it) behaves like the unified memory of the simulation testbench.
- **Word indexing.** A word image streams at byte addresses `4*k` for word *k* (the delivery testbench computes `byte_addr = word_index << 2`). Core fetch and data addresses are always word-aligned, so the core-visible window of each array is 4096 words (16 KB), indexed by `addr[13:2]`; the loader's arrays are 16384 words (64 KB) each, so AXIL traffic can additionally address the unaligned sub-word slots (`addr[1:0] != 0`) that the core never generates.
- **Aliasing.** Because the index is `addr[13:0]`, byte-address bits [15:14] are not decoded: the 64 KB byte-address window of each region aliases onto the array every `0x4000` bytes. Images must fit the 16 KB core-visible window per region (see Known Limitations, Chapter 6).

- **Address windowing.** The core presents full 32-bit addresses; only bits [16:14] region/array bits and [13:0] index bits are decoded. The core's `RESET_ADDR` must be chosen so its used range lands in the window (the loader testbench boots battery images linked at `0x8000_0000` with `RESET_ADDR = 0x8000_0000`, whose low 18 bits fall in the imem region).
- **Per-byte strobes.** Loader writes merge `fub_wdata` per `fub_wstrb` byte; core stores merge `dmem_wdata` per `dmem_wstrb` byte — the same rotated-strobe contract as the simulation memory.

10.6 CTRL Register and Run Control

CTRL register (AXIL offset `0x0002_0000`)

Bits	Field	Access	Description
[0]	`run`	R (set by write)	0 = load mode (power-on default): loader owns both RAM write ports, `core_rst_n` is low; 1 = run mode: the core owns the write ports, `core_rst_n` is high. Sticky until reset — writing 0 does not clear it

`core_rst_n` is literally `run_q` inverted in role: it *is* the run bit. A CTRL write takes effect on the write commit cycle (W-channel fire); the core starts fetching `RESET_ADDR` the very cycle the run write commits.

10.7 Mutual Exclusion (Load Mode vs Run Mode)

- **Load mode (`run == 0`):** the loader owns the RAM write ports; `ld_wr_fire = w_fire && !addr[CTRL] && !run_q` gates image writes. The core is held in reset (`core_rst_n = 0`), parks its fetch at `RESET_ADDR`, and retires nothing.
- **Run mode (`run == 1`):** the core owns both write ports (`core_wr_fire = run_q && dmem_req && (dmem_wstrb != 0)`); loader writes are dropped in logic — the B response still returns OKAY, but the array does not change (testplan scenario ML-09 pins this: an AXIL write to the halted core's ECALL word must not poison the fetch).
- **No arbiter.** Both clients merge into each array in one clocked process; protocol guarantees they are never both entitled at once.

10.8 Programming Sequence

1. Assert `aresetn` low, then release it. The loader powers into load mode: `CTRL` reads 0, `core_rst_n` is low, the core is held.
2. Stream the program image over AXIL writes: instruction words to the imem region (byte offsets from `0x00000`), data (including `.tohost`/`.data`) to the addresses the image expects — with the unified map, data linked above `0x10000` lands in the dmem region automatically. Writes may be full-word or per-byte (`wstrb`).
3. (Optional but recommended) Read back critical words over the AXIL read port; reads work in both modes and see loader and core writes.
4. Write `CTRL.run = 1` (AXIL write of `0x0000_0001` to `0x0002_0000`). The core leaves reset immediately and fetches `RESET_ADDR` on the commit cycle — have any trace sampler armed *before* this write.
5. Observe execution: the core runs to its ECALL halt (`halt` rises with cause `0x1`, `gp == 1` on a successful riscv-tests-style program). Memory the core stores is readable back through the AXIL port in run mode (scenario ML-06).
6. To load a new image, reset the loader (`aresetn`): `run` clears, load mode re-enters.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 Performance Characteristics

11.1 Cycles per Instruction

CPI by access class

Case	Cycles	Notes
Any aligned instruction, including loads/stores that stay within one word	1	CPI 1.0; taken and not-taken branches cost the same
Cross-word misaligned load/store (`addr[1:0] + size > 4`)	2	One retry cycle; still one retired instruction (CPI 2.0 for that instruction)
Halt	0 extra	The halting instruction retires on its own cycle as the trap beat; the hold afterwards is not execution

CPI 1 is per instruction, and the single-cycle discipline taxes every instruction at the rate of the slowest path (below). Code with unaligned data averages slightly above 1.0 — the battery's `ma_data` and `ld_st` programs exercise exactly that case.

11.2 Critical Path Analysis

In a single-cycle design, the clock period must span the slowest instruction's path through *every* block, because every block sits between the same two edges. There is no clock frequency at which this machine is fast, only a frequency at which it is correct; that frequency is set by the longest row below — almost always the load, which crosses both memories and the datapath between them.

Critical-path content per instruction class

Instruction class	Path the clock must span
ALU (OP/OP-IMM)	imem + decode + regfile + ALU + writeback setup
Branch	imem + decode + regfile + comparator + next-PC mux + PC setup
Load	imem + decode + regfile + ALU + dmem + rotate/sign-extend + writeback setup
Store	imem + decode + regfile + ALU + dmem + strobe setup
JAL/JALR	imem + decode + regfile (JALR) + add + next-PC mux + PC setup
Cross-word L/S	all of load/store, plus the retry cycle (two clocks, by design)

Design consequence for integrators: timing closure must be proven against the load row with the *actual* memory macros. Synchronous-read RAM (block RAM) on either port breaks the single-cycle contract outright; the loader honors the contract with `(* ram_style = "distributed" *)` arrays.

11.3 Resource Sketch

An honest sketch, from the RTL rather than synthesis:

Resource sketch (no committed synthesis numbers at v0.1)

Resource	Content
State (core, outside the register file)	Five state elements: `pc` (32b), `retire_count` (64b), `halt_q` (1b), `ls_retry` (1b), `ls_rdata_lo` (32b) — the entire sequential content besides the register file
Register file	32 x 32 bits, 2 combinational read ports, 1 synchronous write port; inferred as distributed RAM/LUTRAM
Decode	One `unique casez` truth table keyed on `{opcode, funct3, funct7}` — no FSM, no microcode, no control ROM
Datapath	ALU (10 ops), immediate generator, source muxes, writeback mux, next-PC mux, dedicated branch comparator, byte-lane rotators
Loader addition	Two 16K-word distributed-RAM arrays (2 x 64 KB), AXIL leaf slaves with skid buffers, one run bit and two write-side holding registers

No LUT/FF synthesis numbers are claimed in this revision; the core's small fixed state and absence of pipeline registers are the structural facts that bound its area.

11.4 Throughput Context

One instruction per clock at frequency f. The suite's rung 2 (merlin, a five-stage pipeline) exists precisely because this machine's clock is capped by the sum of all stages rather than their slowest member — kestrel trades every fast instruction's speed for total visibility. For the education and small-control use cases kestrel targets, that trade is the point.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

12 Integration Guide

12.1 Embedding Recipe (No Loader)

1. **Compile the closure.** Consume `rtl/filelists/kestrel_all.f` (see the filelist table below). It pulls the package, all five FUBs, and the core top.
2. **Provide two memories honoring the contract.** 32-bit-wide, combinational read, per-byte write merge on the data port (`dmem_wstrb`). The fetch port has no strobes — reads only.
3. **Tie the control surface.** One clock, one active-low reset. Set `RESET_ADDR` to the image's entry address.
4. **Observe.** Route the RVFI channel and `halt`/`halt_cause` to your trace fabric; they are the defined observability surface.

12.2 Embedding Recipe (Loader / Board)

1. **Compile the closure.** `kestrel_all.f` includes the loader; it composes the repo's `axil4_slave_wr`/`axil4_slave_rd` leaf slaves through their own filelists.
2. **One clock domain.** `aclk == clk`, `aresetn == rst_n`. The core runs on the loader's `core_rst_n`, not on `rst_n` directly — the loader is the reset sequencer.
3. **Connect an AXIL master** (host CPU, DMA, or debug port) to `s_axil_*` and follow the programming sequence in Chapter 4: reset → stream image → (optional readback) → write `CTRL.run = 1` → run to halt.
4. **Set `RESET_ADDR`** to the image link base; the used address range must fall in the loader window (low 18 bits inside the imem/dmem regions).
5. **Sample before run.** The core fetches `RESET_ADDR` the cycle the `CTRL.run` write commits — arm any RVFI trace capture before issuing it.

12.3 Filelists to Consume

Filelists (`KESTREL_ROOT` = the `kestrel-rv32i` directory; `REPO_ROOT` = the RTLDesignSherpa checkout root)

Filelist	Contents	Environment
`rtl/filelists/kestrel_pkg.f`	The package alone (enums, halt causes)	`$KESTREL_ROOT`
`rtl/filelists/fub/*.f`	One filelist per FUB (alu, decode, imm_gen, regfile, mem_loader), each a complete closure including the package and shared includes	`$KESTREL_ROOT`, `$REPO_ROOT` (loader pulls `rtl/amba` leaf slaves and `reset_defs.f`)
`rtl/filelists/top/kestrel_core.f`	The core top plus its FUB closures	`$KESTREL_ROOT`, `$REPO_ROOT`
`rtl/filelists/kestrel_all.f`	Master list: `-f` of everything above; the whole-area compile closure	`$KESTREL_ROOT`, `$REPO_ROOT`
`dv/filelists/kestrel_tb.f`	The two simulation TB tops on top of `kestrel_all.f`	`$KESTREL_ROOT`

The filelists are the supported compile interface; do not hand-list sources. Both environment variables are registered by the repo's DV framework (`bin/filelist_registry.py`) and by the simulation Makefiles.

12.4 Verification Hooks

kestrel was built RVFI-first: the retire port is the defined observation channel, present from the first vertical slice.

Observability hooks

Hook	Signals	Use
RVFI retire channel	17 `rvfi_*` ports	Full-field diff against a golden model per retired beat; the basis of the cocotb harness and riscv-formal
Halt observability	`halt`, `halt_cause`	Detect program end, classify the cause, sample the post-halt hold
Data-port watch	`dmem_req/addr/wstrb/wdata`	tohost-mailbox watching on the store port; memory-contract checks (rotated strobes, retry addressing)
Loader taps	`core_rst_n`, `o_dbg_busy_wr`, `o_dbg_busy_rd`	Load-mode isolation and AXIL backpressure checks

The V&V cross-reference is the six testplans in `dv/testplans/`: `kestrel_core` (18 scenarios, including the 42-test rv32ui battery, spike lockstep, constrained-random fuzz, and coverage closure), `kestrel_mem_loader` (ML-01..ML-09, including the golden battery over AXIL), and per-FUB plans for `kestrel_alu`, `kestrel_decode`, `kestrel_imm_gen`, and `kestrel_regfile`. Independent of simulation, riscv-formal runs 42 bounded-model-checking proofs (37 instruction models + 5 consistency checks) against the RVFI port — 42/42 PASS at depth 6. Regeneration commands are recorded in each testplan.

12.5 Known Limitations

1. **No trap delivery.** All four spec-mandated trap events halt the core instead; recovery requires reset. Real traps arrive at rung 3 (peregrine).
2. **CSR stub is not CSR support.** Reads return zero, writes drop; anything needing real CSR state (trap handlers, counters, feature probing) is out of contract.
3. **No interrupt inputs, no debug mode, no MMU/PMP.**
4. **Cross-word accesses are not atomic** and occupy two cycles; RVWMO is satisfied vacuously, not by ordering hardware.
5. **No store-to-fetch coherence in the core.** Self-modifying code requires a memory system that unifies the ports (the testbench and the loader both do, by construction).
6. **Loader image window.** Core-visible image data is limited to 16 KB per region (word-indexed slots of a 64 KB array), the byte map aliases every `0x4000`, and the module header's `addr[15:2]` wording describes intent — the implemented index slice is `addr[13:0]` on every port, which is the convention this specification documents.
7. **Loader regions are exactly 64 KB + 64 KB.** Images larger than the window, or linked so their active range exceeds the low-18-bit window, will not load correctly.
8. **Power-up memory content is undefined in hardware** (the simulation zero-initializes for determinism); always load an image before run.

Last Updated: 2026-10-07

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 KESTREL-RV32I Micro-Architecture Specification Index

Version: 0.4 **Date:** 2026-10-20 **Purpose:** Block-level micro-architecture specification for the kestrel-rv32i single-cycle RV32I core

---

13.1 Document Organization

Note: All chapters linked below for automated document generation.

13.1.1 Front Matter

- Document Information

13.1.2 Chapter 1: Overview

- Design Philosophy and Machine Organization

13.1.3 Chapter 2: Functional Blocks

- kestrel_core (top)
- kestrel_decode
- kestrel_imm_gen
- kestrel_alu
- kestrel_regfile
- kestrel_mem_loader
- kestrel_mem_bram

13.1.4 Chapter 3: Internal Interfaces

- Package Types and Top-Level Wiring

13.1.5 Chapter 4: Micro-Architecture Behavior

- The Single Cycle, Step by Step
- Cross-Word Misaligned Retry
- Halt, IALIGN, and the System Layer
- RVFI Aggregation

13.1.6 Chapter 5: Verification Hooks

- Verification Hooks and Observability

13.1.7 Chapter 6: Configuration

- Parameters, Defines, and Filelists

13.2 Related Documentation

- **KESTREL HAS** - Hardware Architecture Specification (external view)
- **Simplified RV32I: kestrel** - Tutorial-study book this core is documented by
- **RISC-V Falcon Suite README** - Suite overview and rung ladder
- **riscv-formal rvfi.md** - Retire-channel field semantics
- **Testplans** - V&V cross-reference (six plans)

Last Updated: 2026-10-07 **Maintained By:** RTL Design Sherpa Project

14 Simplified RV32I: kestrel

Source: The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (riscv-spec.pdf, chapter-cited; this book cites chapters and sections, not pages) and the falcon-suite design documentation **Edition:** Simplified for the kestrel-rv32i single-cycle core, rung 1 of the falcon suite. Derived from condensed study notes; emphasis is on what the as-built kestrel core actually does and verifies, with the RISC-V specification cited as the authority. **Version:** 0.2 **Last Updated:** 2026-10-20 **Status:** Draft. Personal study notes.

14.1 Overview

RV32I is the RISC-V base integer instruction set: thirty-seven instructions, six instruction formats, thirty-one general-purpose registers plus the hard-wired zero register, and a program counter. It is small enough to hold in your head and rich enough to run real compiled code. That combination makes it the canonical teaching ISA — and the foundation rung of the falcon suite.

kestrel is the suite's rung-1 core: a single-cycle RV32I implementation in which one instruction retires per clock and the whole machine — fetch, decode, register read, execute, memory access, writeback — completes inside that clock. There is no pipeline, no hazard logic, no trap machinery, and almost no control state. What there is, is a datapath you can trace by hand and a decode truth table you can read top to bottom.

This book condenses RV32I into the material needed to understand kestrel: the instruction set itself (Chapter 2), the datapath (Chapter 3), the control truth table (Chapter 4), the memory contract including the hardware-misaligned policy (Chapter 5), the verification story — the rv32ui battery, spike lockstep, and the riscv-formal proof that caught a bug forty-two directed tests missed (Chapter 6) — and what the single-cycle discipline costs, which is exactly what motivates rung 2, merlin (Chapter 7).

14.2 Document Structure

14.2.1 Chapter 1: Introduction

- 01_introduction.md - What this book is, what kestrel is, how to read the as-built core
- 02_the_falcon_ladder.md - The four-rung suite, why kestrel is rung 1, what each rung teaches
- 03_conventions.md - Notation, register names, halt-versus-trap terminology, citations

14.2.2 Chapter 2: The RV32I ISA Quick Reference

- 01_formats.md - The six instruction formats and how immediates are assembled
- 02_the_instruction_set.md - The thirty-seven base instructions, group by group
- 03_system_layer.md - What kestrel does with FENCE, ECALL/EBREAK, MRET, and the CSR class

14.2.3 Chapter 3: The kestrel Datapath

- 01_overview.md - Block-level tour, the module list, the complete state inventory
- 02_a_cycle_in_detail.md - One clock, end to end: fetch, decode, execute, writeback
- 03_control_state.md - The five state elements and why there is no FSM
- 04_rvfi.md - The RVFI retire interface and how kestrel aggregates it

14.2.4 Chapter 4: The Control Truth Table

- 01_the_control_bundle.md - The thirteen decode outputs and the default bundle
- 02_the_decode_table.md - The full decode case as a table, every row

14.2.5 Chapter 5: The Memory Contract

- 01_the_contract.md - The Harvard interface, word-wide buses, combinational reads
- 02_misaligned_data.md - The hardware-misaligned decision: rotation, the retry cycle, RVFI encoding

- 03_ialign_and_halt.md - IALIGN=32, the instruction-side misaligned halt, the four halt causes

14.2.6 Chapter 6: Verification

- 01_testbench_and_golden.md - The cocotb testbench, sampling contract, golden interpreter lockstep
- 02_rv32ui_battery.md - The 42-test rv32ui-p-* battery, results table, verdict protocol
- 03_spike_lockstep.md - Spike 1.1.0 lockstep, the misaligned-build note, stream-diff details
- 04_riscv_formal.md - The riscv-formal flow: wrapper, checks, depths, 42/42
- 05_the_cex_story.md - The misaligned-jump counterexample: a real bug the battery missed

14.2.7 Chapter 7: What kestrel Costs

- 01_what_kestrel_costs.md - CPI 1 and its price: the whole machine on the critical path
- 02_merlin_teaser.md - The five-stage answer and what carries over unchanged

---

14.2.8 Version History

Version history

Version	Date	Change
0.1	2026-10-06	First draft
0.2	2026-10-20	Section (h2) headings now open a new page, the pipeline default. No technical content changed.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

15 RISC-V Falcon Suite

Five independent, clean-slate RISC-V cores whose primary product is **education**, in the same spirit as the pumice/scoria/andesite memory-controller line. Each rung is a complete, verified, documented design small enough to hold in your head; the rungs ascend in microarchitectural complexity so the *diff between neighbors is the lesson*. Rungs do not share RTL — each core is the cleanest possible example of its level, not the cheapest to produce. Educational benefit over IP; no rung chases performance or benchmarks.

Design spec (source of truth for scope, ISA, verification, packaging): `docs/superpowers/specs/2026-10-06-riscv-falcon-suite-design.md`.

15.1 The ladder

Each rung carries a raptor-codename, real falcons first (species in strict size order, tracking complexity). The falcon run ends at gyrfalcon — the largest living species — so the top rung steps deliberately to mythology: garuda, the giant divine raptor, king of birds, one past the largest falcon. The codename is the RTL identifier prefix and the module/package prefix; the DIRECTORY is the compound `<codename>-<isa>` form — `kestrel-rv32i`, not `kestrel`. The bare codename is how the core is referred to in prose ("kestrel sees the whole machine in one cycle").

Directory	Codename	ISA	Microarchitecture	Status
`kestrel-rv32i/`	**kestrel**	RV32I	single-cycle	RTL complete; rv32ui battery 42/42 with spike lockstep; riscv-formal 42/42; doc book v0.1 published; board loader sim-verified
`merlin-rv32i/`	**merlin**	RV32I	5-stage in-order pipeline	Planned; structure only
`peregrine-rv32im/`	**peregrine**	RV32IM	advanced in-order: predictor, traps, L1 caches, AXI master	Planned; structure only
`gyrfalcon-rv32ima/`	**gyrfalcon**	RV32IM baseline; gyrfalcon-A flavor RV32IMA	out-of-order capstone: rename, ROB, reservation stations; two flavors — baseline and gyrfalcon-A (`A_EXT=1`:	Planned; structure only

Directory	Codename	ISA	Microarchitecture	Status
			LR/SC + AMO atomics study)	
`garuda-rv32imaf/`	**garuda**	RV32IMAF	gyrfalcon's OoO plus an FPU: FP register file + rename, fcsr precise flags, non-pipelined long-latency units	Planned; structure only (penciled in; beyond the 2026-10-06 design spec, which pins four rungs)

15.1.1 Moved: hive-serv

hive-serv — the SERV-based control/monitoring system (1 VexRiscv control plane + 16 SERV monitor cores) — left this area 2026-10-13 for `compute-eng-ip/hive-serv`: it *uses* RISC-V cores rather than teaching one, and it is the control half of the compute family (its other half is the cell CGRA), not a falcon-ladder rung. This directory is the ladder again, nothing more.

15.1.2 Tetrad engine: gatling-rv32itetrad

gatling-rv32itetrad/ is not a ladder rung either — it is the compute family's tetrad engine (owner PRD 2026-10-17): a barrel RV32I core with the Xcell extension (CFG/POP/PUSH/ECTX, custom-0), one engine per tetrad with four contexts — the four cells — sharing one F/D/X/W datapath. It replaces the cell PRD's ALU + execution-FSM block ("the program is the config") and answers the kestrel-for-the-compute-FSM question in kestrel's own lineage: same RV32I contract and lockstep flow, inverted pipeline lesson — N contexts hide one shared datapath's latency instead of stages hiding latency for one stream. Open decisions and its interface to the bf16 alu-infer block are tracked in vault TASK-001.

15.2 Goals per rung

kestrel — see everything. Single-cycle RV32I: PC, register file, ALU, immediate generator, control as an explicit decode truth table, a plainly documented combinational-read memory contract. The hovering falcon: the whole machine is visible in one cycle — including its cost, since the critical path *is* the machine, which is exactly the motivation for merlin. **Delivers:** the ISA (R/I/S/B/U/J formats, the full base integer set), the datapath, honest control logic.

merlin — flow it. The classic IF/ID/EX/MEM/WB pipeline on the deliberately identical RV32I ISA, so students diff *behavior*, not ISA. Forwarding from EX/MEM and MEM/WB with explicit priority, load-use interlock stall, branches resolved in EX predict-not-taken with flush. **Delivers:** pipeline registers, RAW hazards, forwarding priority, the load-use bubble, control-hazard flushes, why branch resolution stage matters.

peregrine — speed it up. Advanced in-order RV32IM: iterative multicycle multiply/divide (the structural-hazard lesson), BTB + 2-bit bimodal prediction with mispredict recovery, machine-mode CSRs with precise exceptions and CLINT-shaped interrupts, split blocking write-back L1 caches behind an AXI4 master on the repo fabric with MonBus visibility. **Delivers:** speculation and recovery, precise exceptions, the memory hierarchy, bus protocol — and ties the whole repo together (memory controllers, cache IP line, AMBA fabric).

gyrfalcon — hide everything. Out-of-order capstone in two flavors (added 2026-10-11): the baseline — peregrine's RV32IM exactly — and gyrfalcon-A (`A_EXT=1`): LR/SC reservations, store-conditional failure, and AMOs, the atomicity study — so every peregrine battery still carries in both. Dual-issue dispatch, explicit rename map + free list, 32-entry ROB with in-order commit, three reservation stations, wakeup/select over common data buses, speculative dispatch with checkpoint-restore rename rollback. **Delivers:** register renaming and WAR/WAW elimination, the ROB as an undo log, out-of-order completion with in-order retirement — the honest, complete OOO story — and the atomicity study: reservations, SC failure, AMOs, acquire/release as memory-model operations.

garuda — float it. The RV32IMAF delta on gyrfalcon's OoO: a 32-entry FP register file with its own rename, an `fcsr` whose five accrued exception flags (NV/DZ/OF/UF/NX) stay precise-exception-clean through the ROB, and the repo's ieee754 fp32 arithmetic behind the reservation stations — multi-cycle FMUL/FADD and the non-pipelined Goldschmidt FDIV / Newton-rsqrt FSQRT (9–11 cycle class) as the long-latency-unit lesson. Rounding hard-wired RNE; any other requested mode raises NV, keeping the CSR story honest without a dynamic rounding-mode datapath. **Delivers:** long-latency execution scheduling, FP architectural state, and the F-extension ISA delta (F regs, `fcsr`, FMV/FCVT/FSGNJ/FCLASS) — consuming the math library's `math_ieee754_2008_fp32_*` blocks end-to-end.

15.3 Packaging (per rung, the MC-line treatment)

- **Doc book** — *Simplified RV32I: kestrel* and successors, Memory Notes branding, built through `bin/md_to_docx.py` to DOCX/PDF, following the Simplified DFI books' precedent. Each book lives in its rung's `docs/`; the kestrel edition is published at `kestrel-rv32i/docs/simplified_rv32i/`.
- **Formal** — riscv-formal-style ISA compliance where it pays: full on kestrel, retire-interface on merlin, targeted properties (exception precision, recovery, in-order commit, deadlock freedom) on peregrine/gyrfalcon.
- **Golden model** — spike ISS lockstep in the rds-dv cocotb framework; the classic `rv32ui`/`rv32um` riscv-tests batteries as the compliance gate.
- **Drill app** — `bin/apps/core_drills` (ddr_drills architecture): pipeline stage-viewer with forwarding arrows, hazard quiz, branch-predictor drill, and a rename/ROB viewer. App design gets its own doc once merlin's RTL is fixed.

15.4 References and layout

Common primary sources (the RISC-V ISA manuals) live in `references/` and are cited by chapter, not page.

Per-rung skeleton follows the dma-ip/stream component layout convention: `rtl/`, `docs/`, `dv/` (cocotb + lockstep testbenches). Shared suite doctrine lands in `docs/` as it is written. Per-rung `PRD.md`/`CLAUDE.md` arrive with each rung's implementation plan.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

16 kestrel-rv32i Testplans

YAML testplans mapping every kestrel FUB and the core top to the cocotb/pytest tests that exercise them. Format mirrors the pumice/retro_legacy_blocks testplan convention so the same coverage rollup applies (`bin/cov_utils/functional_coverage_tracker.py`).

16.1 Inventory

Testplan	Module	Test file(s)	Scenarios	Verified	Not impl.
`kestrel_regfile_testplan.yaml`	kestrel_regfile.sv	test_kestrel_regfile.py	5	5	0
`kestrel_alu_testplan.yaml`	kestrel_alu.sv	test_kestrel_decode.py	5	5	0
`kestrel_imm_gen_testplan.yaml`	kestrel_imm_gen.sv	test_kestrel_decode.py	6	6	0
`kestrel_decode_testplan.yaml`	kestrel_decode.sv	test_kestrel_decode.py	12	12	0
`kestrel_mem_loader_testplan.yaml`	kestrel_mem_loader.sv	test_kestrel_mem_loader.py	9	9	0
`kestrel_core_testplan.yaml`	kestrel_core.sv	test_kestrel_{core,branch,ls,rv32ui,mem_loader,fuzz}.py	18	17	1
Total			**55**	**54**	**1**

Not-implemented scenarios and why:

- **CORE-18** — functional coverage closure / cover model (**Phase C**).

16.2 What each plan covers

- **RF (kestrel_regfile)**: reset-to-zero, write/readback on both ports, read-during-write old-value, x0 discard, independent port addressing (attributed to the core-level golden trace, RF-05).
- **ALU (kestrel_alu)**: exhaustive 10-op x 16-value x 16-value golden-model sweep including shamt=`b[4:0]`, signed/unsigned compare flags. The flags are unconnected inside kestrel_core by design (branch comparator is separate) — verified at unit level only.
- **IG (kestrel_imm_gen)**: I/S/B/U/J immediate assembly and sign extension with directed ±vectors; B/J LSB-zero rule; plus a SEED-driven randomized sweep (625 vectors, opcode+funct3 anchored per format class) against the Python `imm_golden`.
- **DEC (kestrel_decode)**: the 48-encoding golden truth table (37 canonical RV32I + mret + fence/fence.i + ecall/ebreak + all 6 CSR-stub forms), illegal / reserved encodings landing on halt cause 0xF, ecall/ebreak causes 1/2. Halt cause 3 (IALIGN) is decode-invisible by design and lives in the CORE plan (CORE-13).
- **ML (kestrel_mem_loader)**: AXIL window over both 64 KB regions, per-byte wstrb merge, CTRL.run readback/release sequencing, load-mode isolation (core_rst_n hold, parked PC, no retirement), core↔dmem cross-mux both directions, run-mode observability through the AXIL port, AXIL backpressure (bready/rready held low — bvalid/rvalid hold, busy taps, no lost beat), run-mode loader-write rejection (post-CTRL.run poison write leaves the arrays untouched, B still OKAY), and the golden-trace board path (battery images streamed over AXIL with spike lockstep).
- **CORE (kestrel_core)**: ISA classes via golden trace, branches/jumps (incl. IALIGN halt cause 3), loads/stores incl. the cross-word 2-cycle retry, the system layer (FENCE NOPs, ECALL/EBREAK halt+hold, illegal halt, trap-beat shape), RVFI invariants, the 42-image rv32ui battery, spike lockstep, and the CORE-17 constrained-random fuzz (seeded streams vs the interpreter full-field diff plus spike lockstep at every level). Phase C gap (coverage closure) is tracked as not_implemented.

16.3 JUnit naming convention (observed)

Kestrel's pytest wrappers launch one Verilator build per (cocotb testcase, test level) cell; each cell writes a cocotb JUnit XML whose testcase `name` is the **cocotb test function name**, with the pytest module as `classname`:

```
<testcase name="cocotb_test_kestrel_core_focus"
classname="test_kestrel_core" .../>
```

So `test_function` strings in these plans quote the `cocotb_test_*` node names exactly (e.g. `cocotb_test_kestrel_branch_all`, `cocotb_test_kestrel_rv32ui_battery`). They match the tracker by exact name — there are no pytest-parametrized `[case-level]` node names in the cocotb XML.

Results XML layout: the wrappers set `COCOTB_RESULTS_FILE` to `dv/tests/logs/results_<testcase>_<level>.xml`. Both `dv/tests/logs/` and `dv/tests/local_sim_build/` are gitignored — nothing is checked in; every run regenerates them. The per-cell cocotb XMLs also land one level down at `dv/tests/local_sim_build/<testcase>_<level>/<rand>_results.xml` with identical testcase names, so to feed the tracker from a previous local run, flatten its XMLs first:

```
cd projects/components/riscv-ip/kestrel-rv32i/dv/tests
find local_sim_build -name '*_results.xml' -exec cp {} logs/ \;
```

16.4 Rollup

```
# From the repo root, after a regression (or after flattening a
previous
# local run's XMLs into dv/tests/logs/ as above):
python3 bin/cov_utils/functional_coverage_tracker.py \
    --testplans-dir
projects/components/riscv-ip/kestrel-rv32i/dv/testplans \
    --report \
    --results-dir
projects/components/riscv-ip/kestrel-rv32i/dv/tests/logs
```

The tracker exists for per-scenario status verification: it matches each YAML's `test_function` strings against the JUnit XML testcase names and reports whether the referenced cocotb tests actually ran and passed. Its functional-coverage percentage shows 0% for these plans — by design, as in the pumice house: these YAMLs carry no `coverage_points` blocks, so the tool has no cover data to score. The scenario-level rollup for kestrel is each YAML's `implied_coverage` block: `scenario_tracked` counts `status: verified` scenarios and `implied_percentage = tracked / total_scenarios`. Per-scenario `covers_lines` attribution from Verilator `.dat` files is the pumice `update_testplan_coverage.py` workflow and is deliberately not wired in kestrel yet (CORE-18 owns the cover model).

16.5 Running the tests

```
cd projects/components/riscv-ip/kestrel-rv32i/dv/tests
make list                       # discovered test roots
make run-all-func-parallel      # FUNC regression (gate+func cells)
make run-all-full-parallel      # FULL regression (gate+func+full cells)
```

REG_LEVEL semantics: GATE runs gate cells only, FUNC adds func, FULL adds full. The rv32ui battery reads TEST_LEVEL inside the sim (gate = 3-image smoke, func = 42 images + spike lockstep, full = 42 images without lockstep).

16.6 Functional coverage (CORE-18)

Collection is wired through the central `cov_utils` conftest hook (`dv/tests/conftest.py`) — the same pattern every other component uses. To regenerate:

```
make clean-all
COVERAGE=1 make run-all-full
make coverage-report
```

Measured 2026-10-07 across all 60 cells (including the CORE-17 fuzz at full level, 200 streams):

Metric	Result	House target	Status
Line (Verilator)	94.9%	80%	PASS
Line — kestrel_core	100.0%	—	—
Line — kestrel_decode	95.0%	—	—
Line — kestrel_mem_loader	90.1%	—	—

Toggle collection is enabled (`--coverage-toggle`); the house `merge_testlevel_coverage.py` report tracks line only — toggle numbers are ad-hoc from `coverage_data/merged/latest_merged_coverage.dat` via `verilator_coverage`. Known toggle laggards are data-path width toggles exercised thinly by directed tests; the fuzz's random patterns are the planned ratchet. Coverage artifacts (`coverage_data/`, `coverage_reports/`, `local_sim_build/`, `logs/`) are gitignored by design; the numbers above are the committed record, regenerable with the commands in this section.